

Assignable lambdas with capture

Document #: P3963R0 [[Latest](#)] [[Status](#)]
Date: 2026-01-15
Project: Programming Language C++
Audience: EWG-I, EWG
Reply-to: Ruslan Arutyunyan (Intel)
<ruslan.arutyunyan@intel.com>

Contents

- [1 Motivation](#)
 - [1.1 Workaround is unsatisfactory](#)
- [2 Proposal](#)
- [3 Other considerations](#)
 - [3.1 History and semantics](#)
 - [3.2 Move-only types](#)
 - [3.3 Backward compatibility](#)
 - [3.4 Other parallel algorithms vendors](#)
- [4 Implementation experience](#)
- [5 Proposed Wording](#)
 - [5.1 Modify `\[expr.prim.lambda.closure\]`](#)
- [6 References](#)

§ Abstract

This paper proposes making lambdas with captures copy assignable and move assignable when all captured entities are themselves assignable.

§ 1 Motivation

When lambda expressions were introduced in C++11, they were not assignable. [\[P0624R2\]](#) made lambdas without capture assignable. Unfortunately, lambdas with captures have deleted assignment operators, which prevents certain use cases.

One of the examples is interoperability with the C++20 Ranges library that provides powerful abstractions for lazy evaluation pipelines. Range adaptors like `std::views::transform` store callable objects, including lambdas, as part of their state.

Consider the following example:

```
std::vector<int> v = {1, 2, 3, 4, 5};

// This captureless lambda is trivially copyable
auto captureless = [](auto x) { return x + 1; };
static_assert(std::is_trivially_copyable_v<decltype(captureless)>); // ok

// This lambda with capture is trivially copyable
auto with_capture = [y = 1](auto x) { return x + y; };
static_assert(std::is_trivially_copyable_v<decltype(with_capture)>); // ok

// This view is trivially copyable
auto view1 = v | std::views::transform(captureless);
static_assert(std::is_trivially_copyable_v<decltype(view1)>); // ok

// This view is NOT trivially copyable
auto view2 = v | std::views::transform(with_capture);
static_assert(std::is_trivially_copyable_v<decltype(view2)>); // fails
```

In the example above, the lambdas (both with and without capture) are trivially copyable. But while a captureless lambda results in a trivially copyable `transform_view`, a lambda capturing even a simple `int` by value does not. This distinction becomes problematic when targeting parallel range algorithms for heterogeneous execution, as described in the accepted [\[P3179R9\]](#) proposal.

The thing that makes views not trivially-copyable is *movable-box*. It has a non-trivial copy/move assignment operator when the type (in our case, callable) does not model the copyable concept. This is exactly what happens with lambdas with capture.

§ 1.1 Workaround is unsatisfactory

Users can work around this limitation by manually writing function objects:

```
// verbose
struct add_y {
    int y;
    auto operator()(auto x) const { return x + y; }
};

auto view = v | std::views::transform(add_y{1});
static_assert(std::is_trivially_copyable_v<decltype(view)>); // OK
```

This defeats the whole purpose of lambda expressions as a concise way to define inline callable objects and makes it verbose again to work with C++ standard algorithms with hand-written callables like it was prior C++11. Users normally don't favor writing callable objects by themselves until the latter have a significant reusability.

With the [P3179R9] proposal accepted, users should be able to write simple code like below without hand-written callables:

```
auto result = std::ranges::for_each(
    ext::gpu_policy,
    data | views::transform([val](auto x) { return x + val; })),
    callable);
```

One could say that `ext::gpu_policy` does not belong to the C++ standard; however, the implementations are allowed to have implementation-defined execution policies, so the code above could be standard conformant.

§ 2 Proposal

I propose that the assignment operators of a lambda closure type should follow the same rules as the hand-written callable objects: they should behave as if they are not explicitly written (compiler-generated) and then choose the proper behavior based on closure type layout.

In other words:

1. **Copy assignment operator:** If all captures are copy assignable, the closure type's copy assignment operator should be implicitly defaulted. Otherwise, it should be deleted.
2. **Move assignment operator:** If all captures are move assignable, the closure type's move assignment operator should be implicitly defaulted. Otherwise, it should be deleted.

Example with capture by value:

```
int y = 1;
auto f = [y](int x) { return x + y; };
auto g = f;    // OK today - copy construction
g = f;        // Currently ill-formed, proposed to be well-formed

static_assert(std::is_copy_assignable_v<decltype(f)>); // Currently fails,
                                                        proposed to pass
static_assert(std::is_move_assignable_v<decltype(f)>); // Currently fails,
                                                        proposed to pass
```

Example with capture by reference:

```
int y = 1;
auto f = [&y](int x) { return x + y; };
auto g = f;    // OK today - copy construction
g = f;        // Currently ill-formed, proposed to be ill-formed
```

```
static_assert(std::is_copy_assignable_v<decltype(f)>); // Fails, even with
this proposal
static_assert(std::is_move_assignable_v<decltype(f)>); // Fails, even with
this proposal
```

With this change, the motivating example with `transform_view` (and similar views) would work as expected:

```
auto view = v | std::views::transform([y = 1](auto x) { return x + y; });
static_assert(std::is_trivially_copyable_v<decltype(view)>); // Would pass
```

§ 3 Other considerations

§ 3.1 History and semantics

We asked [P0624R2] authors why they didn't add assignment operators to lambdas with capture. The answer was that they just wanted to solve their immediate problem without making the proposal broader. So, they don't see immediate issues with this proposal.

One might argue that assignment to a closure object is semantically questionable; what does it mean to assign one closure to another? The answer is simple. This assignment would do whatever the compiler-generated assignment would normally do for hand-written callables: either be similar to `= default` or similar to be `= delete` if captured objects are references or not assignable. Beyond the main motivation, this change will make lambdas even closer to hand-written callables.

One could ask about a default constructor for lambdas with capture. It is a valid question and we even could potentially say something along those lines: “*if all the captured members are default constructible then the lambda is default constructible or it has a deleted default constructor otherwise*”. However, it would require more exploration and I don't see the immediate use-case for that.

§ 3.2 Move-only types

It could be that move-only types are captured in a lambda. It is also hard to imagine how it would be possible to create a copy of such an object. So, the example below should be valid C++ but probably unlikely:

```
auto f1 = [p = std::make_unique<int>(1)](int x) { return x + *p; };
auto f2 = std::move(f1);
f1 = std::move(f2);
```

Nevertheless, if one adds `static_assert(std::is_move_assignable_v<decltype(f1)>);` it would pass with this proposal. I do not see any particular problems with that; it's just worth noting.

§ 3.3 Backward compatibility

This is a pure language extension. Code that previously failed to compile (due to deleted assignment operators) would now compile successfully. No existing valid code would change behavior.

Code that relies on `std::is_copy_assignable_v<T>` trait (where `T` is a lambda with capture) for SFINAE purposes would see changed behavior. This example does not sound realistic because it would work with regular callables with some non-static data members compared to lambdas with captures. Furthermore, such an example did not prevent us from adopting [P0624R2], thus making captureless lambdas assignable in C++20.

§ 3.4 Other parallel algorithms vendors

I talked to NVIDIA representatives as a parallel algorithms vendor. They also consider the problem in the [Motivation](#) section important and believe that we need to solve it. They don't have objections to this proposal.

It's worth mentioning that there is [P3960R0] proposal that is authored by both Intel and NVIDIA. It approaches the similar problem but covers broader scope, thus it is complementary to this proposal. It's still worth adding lambdas assignability to not pessimize the cases like with `ranges::transform_view` and make them more consistent with other class types.

§ 4 Implementation experience

The implementation is not done yet but is not expected to cause problems according to the compiler implementors I talked with.

§ 5 Proposed Wording

§ 5.1 Modify [\[expr.prim.lambda.closure\]](#)

- ¹⁷ The closure type associated with a *lambda-expression* has no default constructor if the *lambda-expression* has a *lambda-capture* and a defaulted default constructor otherwise. It has a defaulted copy constructor and a defaulted move constructor ([\[class.copy.ctor\]](#)). It has ~~a deleted copy assignment operator if the *lambda-expression* has a *lambda-capture*~~ and defaulted copy and move assignment operators ~~otherwise~~ ([\[class.copy.assign\]](#)).

[*Note*: These special member functions are implicitly defined as usual, which can result in them being defined as deleted. — *end note*]

§ 6 References

[P0624R2] Louis Dionne. 2017-11-10. Default constructible and assignable stateless lambdas.

<https://wg21.link/p0624r2>

[P3179R9] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-05-29.
C++ parallel range algorithms.

<https://wg21.link/p3179r9>

[P3960R0] Mark Hoemmen and Ruslan Arutyunyan. Fix or Remove Sender Algorithm
Customization.

<https://isocpp.org/files/papers/P3960R0.html>